Page No.:
Date:

Assignment No. 1

1. Define the term i] Software ii] Software Engineering.

→ i] Software

Software is a set of instructions, data or program that used to operate computers ~~within~~ and execute specific task.

There are multiple types of software :-

1] System software
2] Application software
3] Real time software
4] Embedded software

ii] Software Engineering

- Software engineering is the process of designing, developing, testing and maintaining a software.
- It is a ~~system~~ systematic and disciplined approach to software development that aims to create high quality, reliable and maintainable software.

2. Explain any two characteristics of software.

→ Characteristics of software :-

1] Software is developed :-

Unlike hardware, software is not produced through a manufacturing process. It is developed using engineering process & design process.

2] Software does not wear out :-

Software does not degrade over time like hardware. Its failure rate is high at the beginning but decreases as bugs are fixed and reach to its stable state.

3. Define the term Extreme Programming.

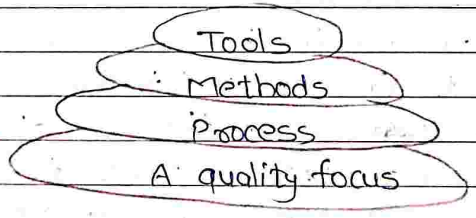
→ Extreme Programming (XP) is an Agile software development model that focuses on high-quality software, continuous customer involvement, teamwork, frequent testing and quick delivery.

Objectives of XP:-

1] Deliver high-quality software.
2] Reduce software errors.
3] Increase customer satisfaction.
4] Deliver software quickly.

4. Draw layered Approach for software Engineering.

→ Layered approach for software engineering:-



1] Quality focus :- Quality should be maintain in every step of software development.
2] Process :- Process tells us what steps to follow to develop software.
3] Methods :- Method tells us how to perform each development activity.
4] Tools :- Tools are software programs that help developers perform their work easily and quickly.

5. Explain Waterfall Model with neat diagram.

→ Waterfall Model:-

1] The waterfall model is defined as 'model of building in stages' where one stage must be completed before the next stage begins.
2] It is referred as linear sequential life cycle model.
3] In this model, process flows from top to bottom and it can't go back in reverse order that's why it is called as waterfall model.
4] In this model, the phase in the development process begins only if the previous phase is completed.
5] Waterfall model is basically used for the project which is small and there are no uncertain requirements.
6] While developing a software by using waterfall model method, if we want to make changes in it, then we have to perform or complete these all phases repeatedly until software is developed.

Phases of waterfall model:-

1] Requirement gathering:-
In this phase, developers collect and understand the customers requirement.

2] Planning:-
In this phase, developers decides how to develop a software (like how much time, budget and how many team members will required).

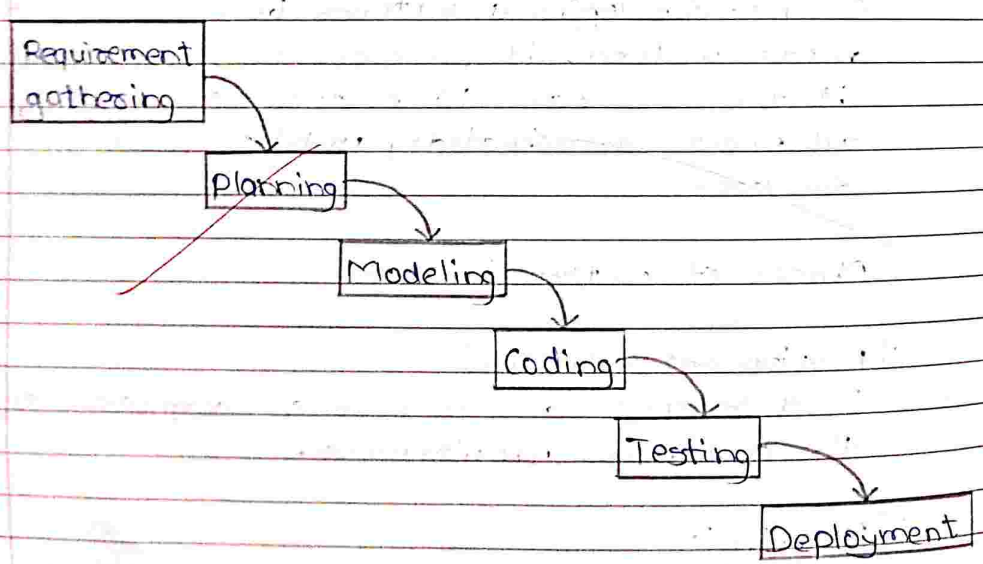
3] Modeling :-
In this phase, developers creates a basic prototype or design of a software before coding starts.

4] Coding :-
The actual coding of a software is done in this phase.

5] Testing :-
Developed software is tested in this phase.

6] Deployment :-
After testing the software, it is delivers to the end user or into the market.

Diagram of Waterfall model :-



6. Explain RAD model with neat diagram.

→ RAD Model :-

1] RAD stands for Rapid Application Development model.
2] The RAD is a software development model in which software is developed by dividing it into small modules and getting continuous feedback from the customers.
3] The RAD is a type of incremental process model.
4] The RAD is a high speed adaption of waterfall model.
5] IBM first proposed the RAD model in 1960's and later ~~it~~ on it was ~~developed~~ introduced by James Martin.
6] Why it's called ~~RAD Model~~ Rapid Application Development :-
- Rapid means fast.
- Application means software.
- Development means building.

Phases of RAD Model :-

1] Communication :-
In this phase, requirement gathering is done means finding the customers needs.

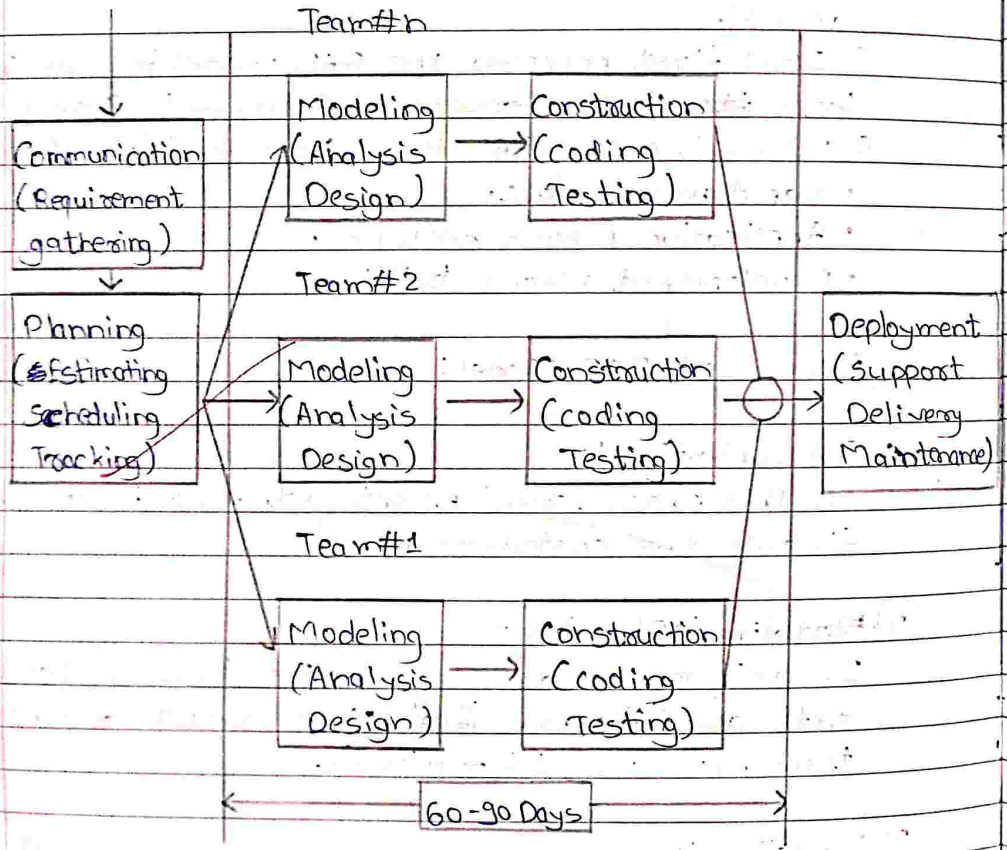
2] ~~Modeling :-~~ Planning :-
In this phase, developers decides how to develop a software (like how ~~many~~ much time, budget and how many team members will required).

3] Modeling :-
In this phase, developer creates a basic prototype or design of a software.

4] Construction:-
The actual coding of a software is done and actually software is developed & tested in this phase.

5] Deployment:-
After testing, software is delivered to the end users or into the market.

Diagram of RAD Model:-



7. Explain adaptive software development w.r.t. i] Speculation ii] Collaboration.

→ i] Adaptive Software Development:-
1] Adaptive Software Development (ASD) is an Agile process model.
2] It focuses on adapting to changing requirements, continuous learning, teamwork and customer feedback.
3] ASD consists of three main phases:- speculate, collaborate and learn.

i] Speculation:-
1] The project is planned based on the available requirements.
2] The plan is kept flexible because requirements are expected to change as the project progresses.
3] Instead of a fixed, detailed plan, the team makes an adaptive, best-guess plan that can be revised later.
4] Example:- A customer wants an online shopping website with: Login
Product search
~~Login~~
~~Search bar~~

ii] Collaboration:-
1] Developers, testers, managers and the customers work closely and continuously together throughout the development.
2] Because of working together suggestions of customers are captured and acted upon quickly.
~~3] Example:- A customer wants an online shopping cart module~~

3] Example:- While developing a shopping cart, customer request to add wishlist feature, due to working closely and continously together, the team dimmediately discuss about it and include it in next iteration.

8. Differentiate between Waterfall Model & Spiral Model.

→

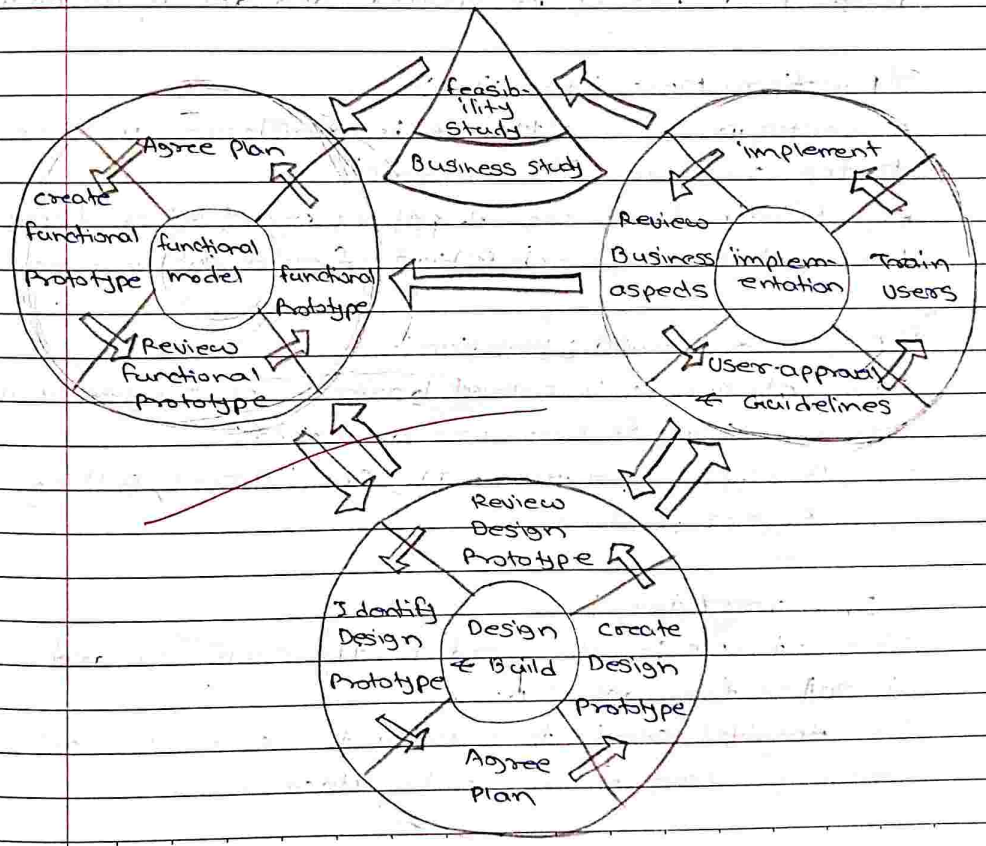
Features	Waterfall Model	Spiral model
Complexity	The waterfall model is simple and easy.	The spiral model is more complex.
Development method	This model work in a sequential method.	This model work in evolutionary method.
Framework	It is based on linear framework type.	It is based on linear and iterative framework type
Cost	It is comparatively less expensive.	It is comparatively more expensive.
Testing	Testing is done after coding phase.	Testing is done after engineering phase.
Adoption	It is adopted by customers.	It is adopted by developers.
Project size suitability	Waterfall model is suitable for small project.	Spiral model is suitable for large project.

9. Explain DSDM.

→ DSDM :-

1] DSDM stands for Dynamic System Development Method.
2] DSDM is an Agile software development model.
3] It focuses on developing software quickly, within budget and according to customers requirements.
4] It emphasizes continuous users involvement, frequent software delivery and teamwork.

Diagram of DSDM :-



Phases of DSDM:-

1] Feasibility study:-
This phase checks whether the project is possible and ~~to deve~~ practical to develop or not.
e.g., Checking if an online banking system can be completed in 6 months within budget.

2] Business study:-
In this phase, developers understand the business requirements and customers needs.
e.g., For Hospital Management System - understanding needs like patient registration and doctor appointment.

3] Functional Model Iteration:-
A prototype of a software is developed and shown to the customers for feedback.
e.g., Building login page & patient registration first, then adding a 'search patient' feature customer requested.

4] Design and Build Iteration:-
The software is improved based on customers feedback and remaining feature are developed.
e.g., Developing and testing the Appointment, Billing and Report modules.

5] Implementation:-
The software is delivered to the customers and installed for actual use.
e.g., Hospital staff start using the system; an online online payment feature can be added now.

Assignment No. 2

1. Enlist core principles of software engineering practice.

→ 1] The Reason It All Exists
2] Keep It Simple
3] Maintain the Vision
4] What you Produce, Others Will Consume
5] ~~Plan~~ Be Open to the Future
6] Plan Ahead for Reuse
7] Think Before You Act

2. Explain following terms with reference to requirement engineering.

→ i] Inception:-
Understand the basic idea of the project and the purpose of the project.
Example:- We want to make online food ordering app.

ii] Elicitation:-
Collect requirements from the users, customers and stakeholders.
Example:- Customers say they want online payment and orders tracking.

3. What is Software Coding? State three principles of code validation.

→ Software coding:-
Software coding is the process of converting a software design ~~in~~ and logic into a programming language to develop a software program.

Three principles of code validation are:-
1] Review the code thoroughly :-
Code should be checked carefully for errors.
2] Perform unit testing :-
Each unit should be tested separately.
3] Refactor the code :-
Improve the code structure without changing its output.

4. ~~Explain~~ Enlist any four planning principles.
→ 1] Understand the scope of the project.
2] Make planning by involving customer.
3] Planning activity should be iterative.
4] Make estimation.
5] Describe changes.
6] Consider risk while defining plan.
7] Be realistic.

5. Explain the general format of software Requirement Specification (SRS).
→ Software Requirement Specification (SRS) :-
SRS is a document that clearly describes all the functional and non-functional requirements of a software system. Including what a system should do and how it should perform.

General format of Software Requirement Specification (SRS) :-

1. Introduction :-
1.1 Purpose
1.2 Document Conventions
1.3 Intended Audience
1.4 Additional Information
1.5. Contact Information / SRS Team Members
1.6 References

2. Overall Description :-
2.1 Product perospective
2.2 Product functions
2.3 User classes & Characteristics
2.4 Operating Environment
2.5 User Environment
2.6 Design / Implementation constraints
2.7 Assumption & dependencies

3. External Interface Requirements :-
3.1 User Interface
3.2 Hardware Interface
3.3 Software Interface
3.4 Communication Protocols & Interface

4. System features :-
4.1 Description & Priorities
4.2 Action / result
4.3 Functional Requirements

5. Others Non-functional Requirements :-
5.1 Performance
5.2 Security
5.3 Safety
5.4 Quality
5.5 Project Documentation
5.6 User Documentation

6. Other Requirements :-
6.1 Appendix A
6.2 Appendix B

6. Describe Communication Practices. (Any 4)
→ Communication Practices :-
1] Listen Carefully :-
i] Listen to the customer while ~~he is~~ explaining the project.
ii] Avoid ~~&~~ interruptions in communication.

2] Prepare before communication :-
Collect require information about ~~#~~ a topic before ~~the~~ meeting the customer.

3] Use face-to-face Communication wherever possible :-
Use face-to-face communication whenever possible because direct communication avoids the misunderstanding.

4] Ask Questions :-
Ask questions whenever a particular point is not clear or having a little bit confusion about that point.

5] Take Notes :-
Write down ~~the~~ all important points or requirements.

6] Use Simple Language :-
Use simple words while communicating with the customer, instead of using difficult or technical words.

7] Stay Focused :-
Discuss only about project-related topics.

8] Confirm Understanding :-
Repeat the requirements of customer to confirm them.

9] Respect Everyone's Opinion :-
Listen to the all team members ideas and opinions.

10] Work Together :-
Communication should encourage teamwork.

8. Define use case. Draw use case diagram for Railway Ticket Reservation System.
→ Use Case :-
1] A use case diagram is a type of Unified Modeling language (UML) diagram that represents the interaction between actors (users or external systems) and a system under consideration to accomplish specific goals.
2] It provides a high-level view of the system's functionality by illustrating the various ways users can interact with it.
Annotation of Use Case Diagram :-
1] Actor - An actor is person that interacts with system.



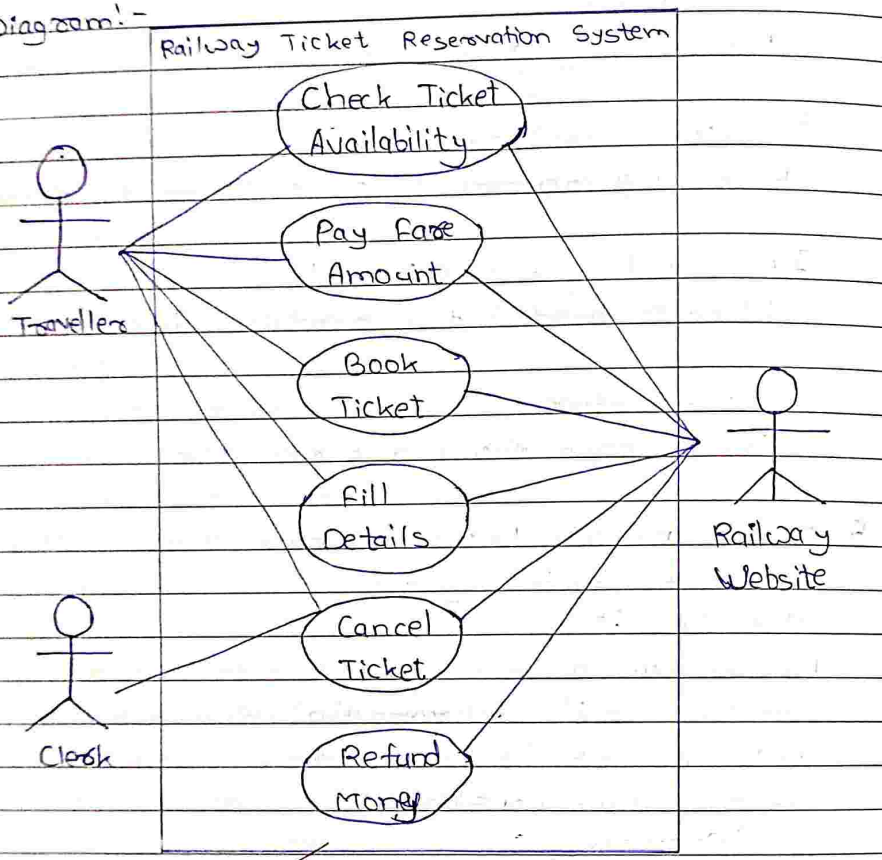
2] Use Case - It represents specific function that system provides to users.



3] System Boundary - It defines what belongs to system and what does not.



Diagram:-



7. Explain Functional and Non-Functional requirements of the software.

→ Functional Requirement:-

1] A functional requirement specifies what a software system should do.

2] It describes the functions, services or operations that the system must perform based on user needs.

3] Functional requirements are easy to capture and the requirements are mandatory.

4] Example:- The system shall allow users to log in using their email and password.

Non-functional Requirement:-

1] A non-functional requirement specifies how well a ~~sf~~ software system should perform.

2] It describes the quality attributes, performance, security, usability, reliability and constraints of the system.

3] ~~Fo~~ Non-functional requirements are hard to capture and the requirements are non-mandatory.

4] Example:- The system shall load the login page within 2 seconds.

9. Explain Software Deployment Principles.

→ Principle 1:

Manage/fulfill customer requirements:

In most case customer wants more than he/she has stated earlier as his requirements or expectations. In many cases customer is disappointed, even after getting all his requirements satisfied. Hence, at the point of software delivery developer must have skills to manage customer's requirements.

Principle 2:

Record-keeping mechanism must be established for customer support:

Customer support is essential and important factor in deployment phase. The support should be well planned and with proper record-keeping mechanism.

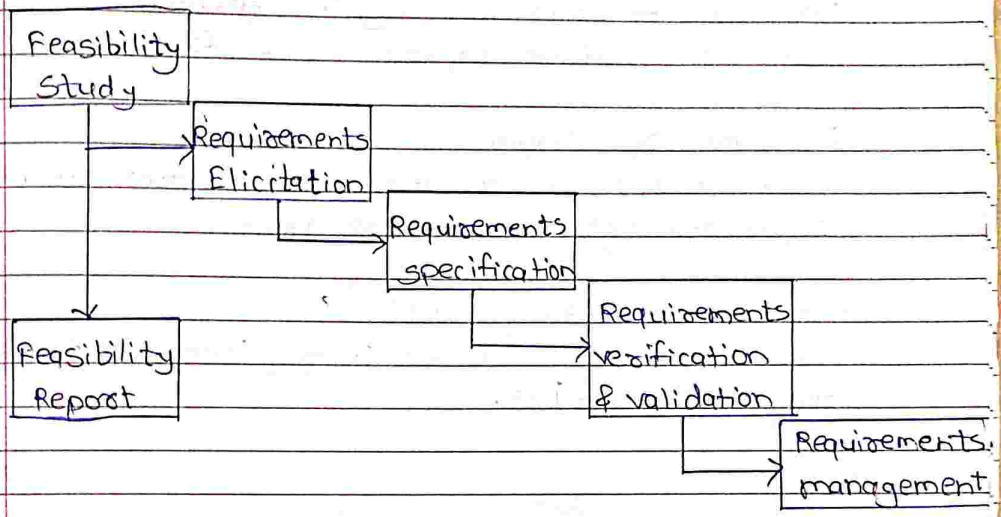
Principle 3:
Provide essential instructions, documentations and manual:
Actual software project delivery must include all documentation, help files and guidance for handling software by user.

Principle 4:
Assemble and test complete delivery package:
i] Necessary supported operational features and help.
ii] Essential manuals required for software troubleshooting.
iii] All executable file of developed software.
iv] Necessary installation procedures.

Principle 5:
Do not deliver any defective or buggy software to the customer:
Software should be properly tested and checked before delivering it to the customer. Software should work according to customer requirements and there should be no errors.

10. Explain Requirement Engineering Process.
→ Requirement Engineering Process:
Requirement Engineering is the process of collecting, analyzing, documenting, validating and manging the requirements of a software system.

Steps Involved in Requirements Engineering:



1. Feasibility Study:-
Feasibility study means checking whether a software project can be successfully developed or not before starting actual work.
Types of feasibility study:-
1] Technical feasibility
2] Economic feasibility
3] Operational feasibility

2. Requirements Elicitation:-
Collect requirements by talking to customers, users and other stakeholders.

3. Requirements Analysis:-
Study and organize the collected requirements and remove confusion or conflicts.

4. Requirements Specification:-
Write all requirements clearly in a document, usually called SRS (Software Requirements Specification).

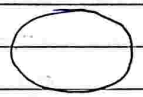
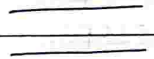
5. Requirements Validation:-
Check whether the requirements are correct, complete, clear and match customer's needs.

6. Requirements Management:-
Manage and control changes in requirements throughout the software project.

Assignment No.3

1. List and draw symbols used in DFD.

→ Symbols used in DFD:-

	Name	Symbol
1]	Process	
2]	External entity	
3]	Data flows	
4]	Database	

2. Define Data Object and Data Attributes.

→ Data Object:-
A data object is a collection of related data or information that represents a real-world entity or concept in a system.

Example:-
A student is a data object. It can contain information such as student name, roll number, branch & marks.

Data Attribute:-
Data attributes are the individual properties or characteristics that describes a data object.

Example:-
For a student data object the attributes are roll no, name, branch & marks.

3. Explain Cardinality and Modality with example.

→ Cardinality:-
Cardinality is the specification of number of occurences of one object that can be related to the numbers of occurences of another object. Cardinality is usually expressed as simply "one" or "many".

What it means: How many things of one type related to many things of another type.

Example:
How many cars can one person own? Maybe 1 or maybe many or how many people can own the same car?

Modality:-
Whether the relationship between two things is mandatory or optional.

Example:
Does a person have to own a car? No, it's optional so the modality would be 0. (optional).

Types of Cardinality:-

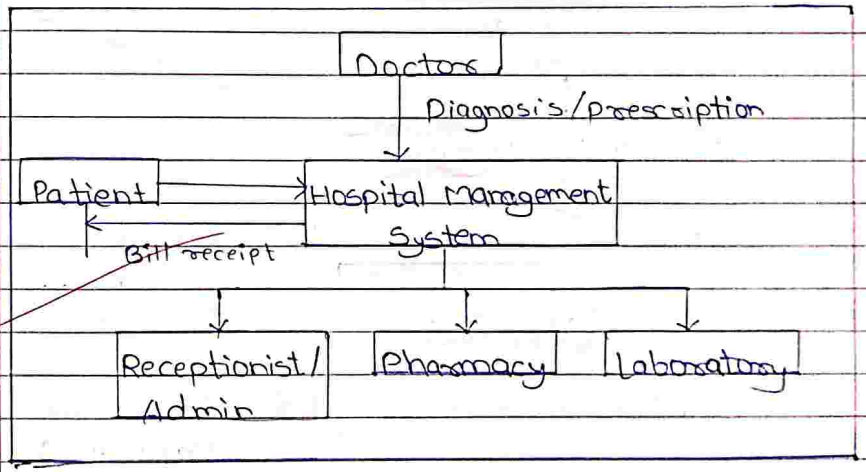
1) One-to-one
An occurrence of object A can relate to one and only one occurrence of object B and an occurrence of object B can relate to one and only one occurence of A.

2) One-to-many (1:M)
One occurrence of object A can relate to one or more occurences of object B but occurence of object B can relate to only one occurence of A.

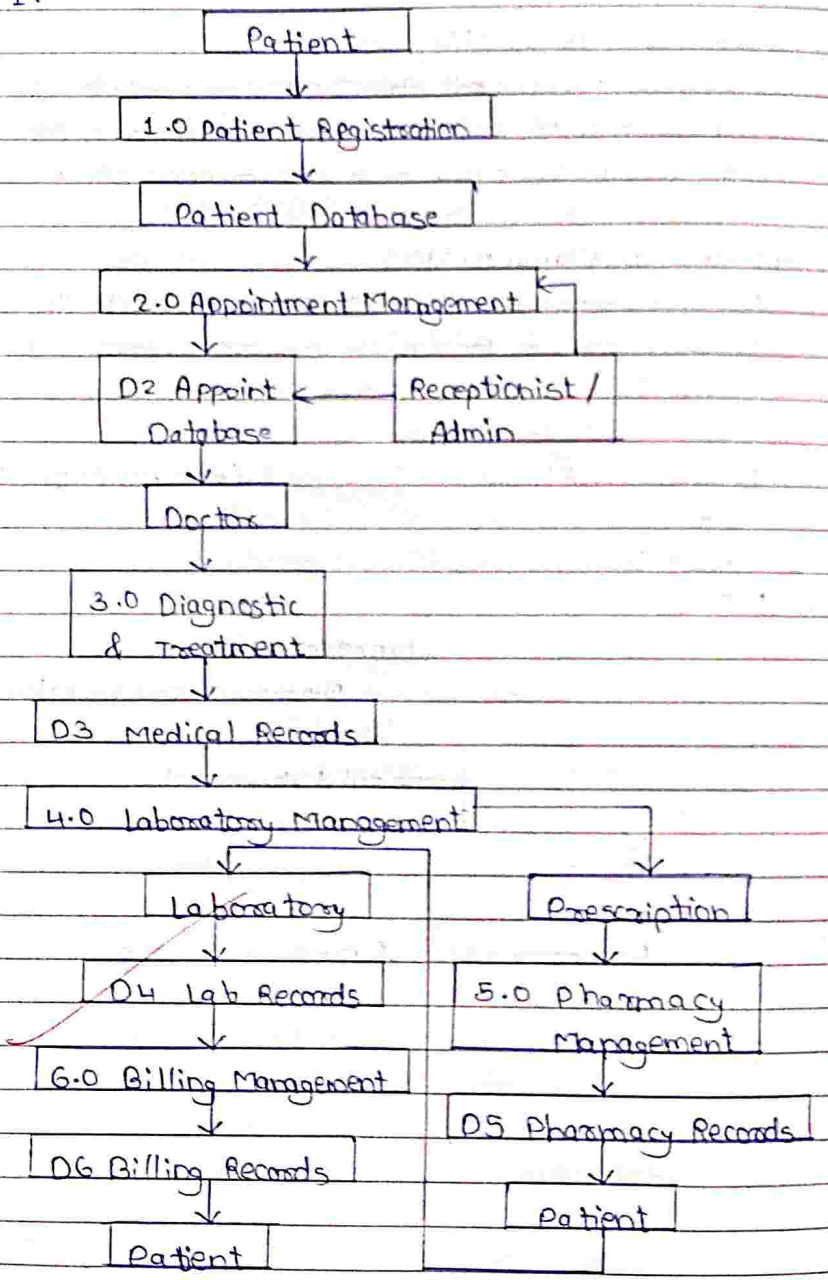
3) Many-to-Many (M:M)
An occurrence of object A can relate to one or more occurences of B, while an occurrence of B can relate to one or more occurences of A.

4. Draw a DFD level 0, level 1 for Hospital Management System.

→ Level 0:

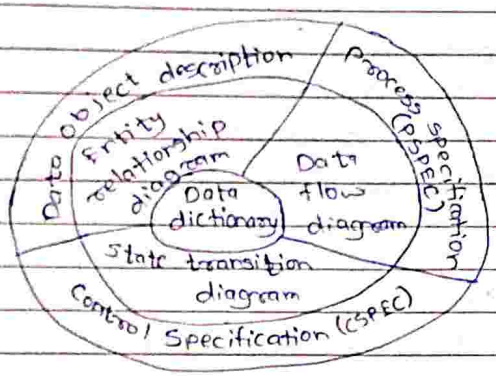


Level 1:

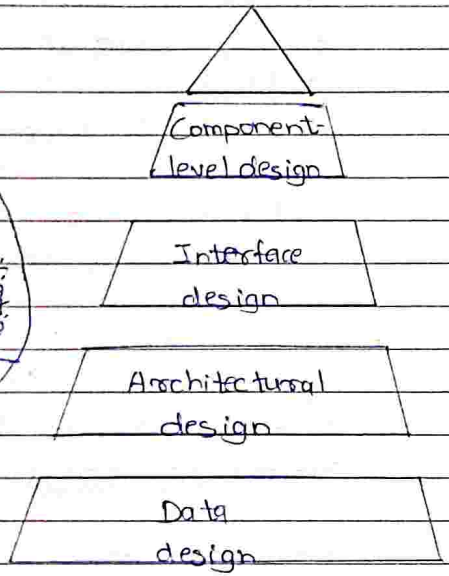


5. Explain the transition analysis model into design model with diagram.

→



The analysis model



The design model

- Design Model :-

1] Data design :-
Data design describes what data the software will store & how that data will be organized.
example :- student ID, student-name, course, marks.

2] Architectural design :-
Architectural design describe the overall structure of software & how its major part are organized.
example :- Architectural design = what are main part of software.

3] Interface design :-
Interface design describes how users and different software components communicate with each other.
example :- In login page : -> username + password -> login Button -> system

4] Component-level design :-
Component-level describes the internal working of each individual component or module of the software.
example :- In login module → Take username & password
check details in the database
if correct → login successful.
if incorrect → shows error.

- Analysis Model :-

1] Data Dictionary :-
A data dictionary is a collection that gives complete information about data used in a software system.
example :-
1] Student_name → name of student
2] Roll_No → Roll number of student

2] Entity Relationship Diagram (ERD) :-
ERD shows the entities and relationship between them in a system.
example :-
Student → Enrolls in → Course
Here, student & course are entities and enrolls in is their relationship.

3] Data Flow Diagram (DFD) :-
DFD shows how data moves from one place to another in a system.
example :- In an ATM:
Customer → Enters PIN → ATM → Bank Database

4] State - Transition Diagram :-
It shows how an object changes from one state to another when an event occurs.
example :- for an online order :
Order Placed → Order Confirmed → order Shipped → Order Delivered

5] Process Specification :-
Process specification explains steps performed by a particular process.
example :- For Login process :
Enter username & password → check details → Correct? →
Login successfully ~~if:~~
if incorrect → show Error Message

6] Control Specification :-
Control Specification explains how system responds when a particular event or condition occurs.
example :- In an ATM : → Wrong PIN entered 3 times → ATM blocks Card.

7] Data Object Description :-
It gives details or properties of a data object.
example :- for student object :
Student : RollNo : 101, Name : Rahul, Course : IT